Can AI help keep data validation rules up to date?

Maintaining and extending expert rules when data standards change

MSc Artificial Intelligence thesis proposal

Tomash Oosterbaan
University of Groningen | Avanade and Rabobank

9 November 2026

Overview

What this thesis is about. Banks and listed companies send financial reports to supervisors in a fixed digital format. Every number in such a report carries a standard name, such as `Assets`, together with a date and a unit, so that software can read it. Before a report is accepted or used, software runs *validation rules*: small programs that check whether the numbers are consistent. A simple rule says that total assets must equal total liabilities plus equity. If a report states assets of $340 million and liabilities plus equity of $350 million, the rule raises an alert and an analyst investigates the difference (Figure 2 in Section 1). Supervisors and industry bodies publish hundreds of such rules, and banks add rules of their own.

Two problems. First, the dictionary of names behind the reports, called the *reporting standard*, changes every year. Names are replaced, removed or added, and report tables get new rows or columns. A rule that still refers to an old name can silently stop checking anything. For example, the 2026 US accounting dictionary replaced the name `BalanceSheetLocationAxis` by `StatementOfFinancialPositionLocationBalanceAxis`, and the committee that maintains the public US rules had to update every rule that used the old name. Today, experts find and update such rules by hand. Second, even up-to-date rules miss some errors. New rules can close these gaps, but a badly chosen rule can flood analysts with alerts on data that is correct.

What the thesis tests. Language models, AI systems that read text and write code, could help with both problems. The thesis tests whether they actually do, in three experiments (Figure 1):

1. **Updating rules (RQ1).** A model receives an old rule and the published description of what changed in the standard, and proposes the new version of the rule. The proposal is compared with the update that the experts actually published.
2. **Adding rules (RQ2).** A model proposes new rules. To test them, known errors are planted in copies of real reports, for example an amount multiplied by 1,000. We count how many planted errors the new rules catch that the existing rules miss, and how many extra alerts this costs.
3. **Testing over time (RQ3).** The updated and new rules are run on real past reports in the order in which they were filed, and on new reports as they arrive during the thesis. We count real errors found, meaning errors that companies later corrected and that were checked by hand, and the number of alerts raised.

Why this can be measured. Two organisations publish their rules together with every earlier version: the XBRL US Data Quality Committee (DQC), which writes rules for US company reports, and the European Banking Authority (EBA), which writes rules for the reports of European banks. The difference between the old and the new version of a rule is an answer written by experts. Nobody in this project decides what the right update is. All data is public; no bank data is used.

What comes out. The thesis delivers a public benchmark of real rule updates, measurements of when AI helps and when simple methods are enough, and a software platform in which every proposed rule is tested and first runs in *shadow mode*: its alerts are recorded but have no effect

on the data. A person then decides whether the rule should only be monitored, should warn, or may block data. Table 1 explains the terms used throughout. Each section is written so that it can be read on its own with this table at hand.

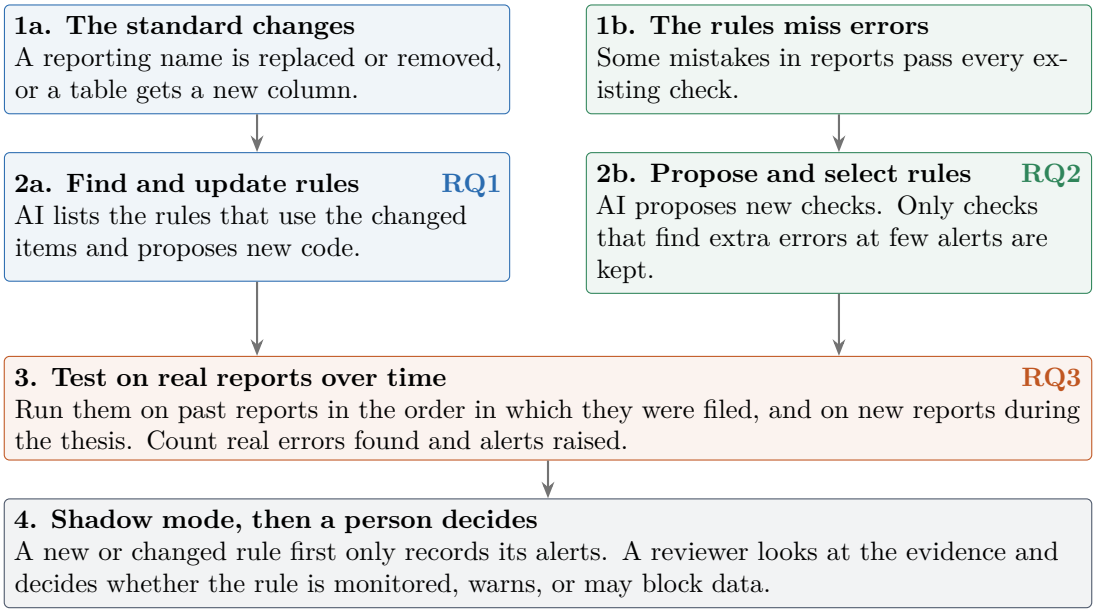


Figure 1: The workflow studied in this thesis. Each research question has an answer key that nobody in the project wrote. RQ1: the update that the experts published. RQ2: errors deliberately planted in copies of reports, at known locations. RQ3: real corrections that companies made later, each reviewed by hand.

1 Problem and research objective

Why validation matters. Data used for financial and supervisory reporting must be accurate, complete and consistent. The Basel Committee's BCBS 239 sets out principles for risk data aggregation and reporting, and the European Central Bank's 2024 guide describes the governance, controls and validation it expects from supervised banks [1, 2]. The ECB's 2024 supervisory report found continuing weaknesses in data accuracy, data architecture and reporting processes [3]. Validation rules are one way to meet these requirements. At Rabobank, technical validation, business rules, monitoring and the manual investigation of exceptions are among the controls in use [4].

What a validation rule does. Figure 2 shows a real rule. DQC rule `DQC_0004` reads two numbers from a report, total assets and total liabilities plus equity, and checks that they are equal when they refer to the same company, date and unit. The DQC documents the rule with an example in which assets are $340 million and liabilities plus equity $350 million. The rule then reports the $10 million difference so that an analyst can investigate [5]. Every rule has these three parts: the facts it reads, the condition it tests and the message it produces. The EBA publishes similar rules for European banks. These refer to numbered cells in standard report tables instead of named facts (Table 2).

How a change to the standard affects a rule. Rules refer to the names, tables, rows and columns defined in a reporting standard. For US company reports, the FASB publishes a new version of the US GAAP taxonomy every year, together with files that list which names are deprecated and which names replace them [7, 8]. For European banks, the EBA publishes successive releases of its reporting framework, with report tables, data definitions and validation rules [9, 10]. When a new version appears, a rule may need one of the four kinds of change in Table 3. The first two can be read from the change files of the standard. The last two require an

Term	Meaning in this proposal
Reporting standard	The dictionary of names that reports must use, with their meaning. For US company reports this is the US GAAP taxonomy (US GAAP are the US accounting rules; a taxonomy is a dictionary of reporting names), published every year by the Financial Accounting Standards Board (FASB). Each yearly version is a *taxonomy year*. For European banks it is the EBA reporting framework: numbered report tables with rows and columns.
Deprecated name	A name that a new version of the standard marks as outdated, often with a named successor. It is usually removed in a later version.
Change files	Files published with each US taxonomy year that list deprecated and removed names and, where one exists, the name that replaces them. A *change notice* is a short summary of these files that is given to a model.
Report, filing	A set of numbers that a company or bank submits to a supervisor. Each number is a *fact*: a name from the standard, a value, a date or period, and a unit.
XBRL	The digital format in which each number in a report is labelled with its name, date and unit.
Validation rule	A small program that checks a report. When the check fails, the rule raises an *alert*. Each DQC rule is one file that contains one or more checks, called *sub-rules*.
DQC	XBRL US Data Quality Committee. Publishes about 180 rules for US company reports, with their code and official test cases.
EBA	European Banking Authority. Publishes the report tables and validation rules for European banks.
Alert, alert budget	An alert is a value flagged for a person to review. The alert budget is the number of alerts per report that a team is willing to review.
Planted error	An error deliberately introduced into a copy of a report, at a recorded location. Used to test whether a rule catches it. A copy without planted errors is a *clean copy*.
Amended report	A corrected version of a report that a company files later. A changed number may reveal an error in the original.
Shadow mode	A new rule runs on real data and its alerts are stored, but it does not stop or change any data.
Monitor, warn, block	The three action levels of a rule: record alerts only; let data through with a warning; hold data for review.
Language model	An AI system that reads text and writes text or code. *Open-weight* models can be downloaded and run on one's own computers, so no data leaves the organisation.
Arelle	Open-source software that runs XBRL validation rules on reports.

Table 1: Terms used throughout the proposal.

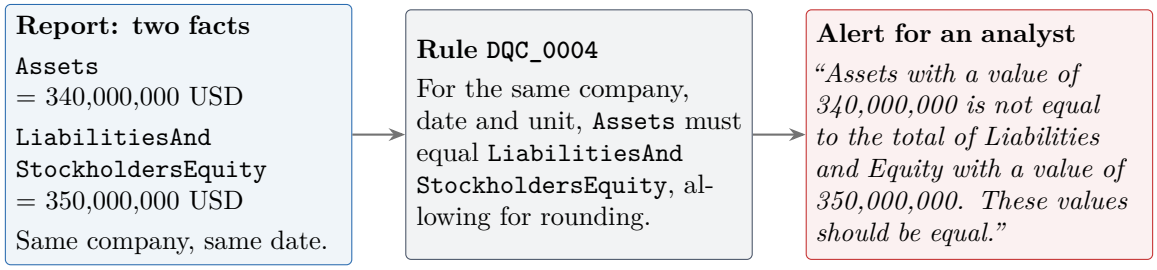


Figure 2: How a validation rule works. The numbers and the message text come from the DQC's documentation of rule `DQC_0004` [5], not from a real company.

understanding of what the rule is meant to check.

How often rules change. The 2026 DQC rule set for US reports contains about 180 rules, made up of 400 sub-rules. A pilot study for this proposal (Section 2.4) compared the DQC versions for the 2023, 2024, 2025 and 2026 taxonomies sub-rule by sub-rule. Over these three annual updates, the committee made 62 changes to the logic of sub-rules and added 56 new sub-rules. Changes that only renamed internal variables or edited the message text are not counted (Section 2.4). In the EBA rule workbooks, the EBA labelled 1,406 changes in 1,094 rules as changes to the formula

Publisher	What the data contains	What a rule checks (simplified)
DQC, for US company reports	Named financial figures such as assets, liabilities and equity, each with a date and a unit	`DQC_0004` checks that assets equal liabilities plus equity. The publisher's example compares $340m with $350m [5].
EBA, for European bank reports	Standard report tables in which each cell is identified by a table number, a row number and a column number	Rule `v6053_m` checks that one cell in table F 01.02 equals the sum of a set of cells in table F 09.01.1 [6] (Section 5.3).

Table 2: Two real kinds of validation rule. The DQC message and the EBA formula come from the publishers; the descriptions are simplified.

Kind of change	Real example	What the rule needs
A name is replaced by a named successor	The 2026 US taxonomy replaced `BalanceSheetLocationAxis` by `StatementOfFinancialPosition LocationBalanceAxis`. DQC sub-rule `US.0164.10087` used the old name [11].	Replace the name.
A name is removed without a successor	The 2025 US taxonomy removed `CashAndCashEquivalentsPeriodIncreaseDecrease ExcludingExchangeRateEffect`. DQC sub-rule `US.0004.9286` referred to it [11].	Remove the reference without breaking the rule.
A report table changes	EBA release 3.0 added column 0065 to the cells summed by rule `v6053_m`; the EBA's note reads "Added column on POCI", for exposures that were already credit-impaired when purchased or originated [9].	Change the formula.
The experts correct their own rule	In a hand-checked pilot sample of 12 DQC changes, three fixed the rule itself: a missing exception, a missing part of a calculation and a bracket error [11].	Correct the logic.

Table 3: Four ways in which a new version of a standard can require a rule to change. All examples are from the published DQC and EBA rule versions examined in the pilot work (Section 2.4).

itself; 266 of these only switched from three-digit to four-digit row and column codes. Formula edits under other labels, such as notation-only updates, come on top of this. Each change is a maintenance task with a published expert answer. These tasks are the subject of RQ1.

What happens to an old rule. This depends on the publisher. The DQC publishes a new version of its rules for each taxonomy year. The EBA distributes revised rule packages, including quarterly packages for corrections, deactivations and changes in severity [12, 9]. An organisation that has written its own rules must itself work out which rules a change to its data definitions affects. The platform proposed here lists the rules that are likely to be affected, suggests a change and tests the result. The project does not assume that Rabobank maintains its rules in the same way as either publisher.

The second problem: missing rules. A set of correct rules can still miss errors, and a data team usually cannot see the mistakes that pass all its checks. It can inspect past corrections, review incidents, read the reporting definitions and test its controls by planting errors in copies of the data. Language models can use these materials to suggest new rules, but a plausible rule can still be wrong. A proposed upper limit on a loan amount, for example, means nothing unless the business has set that limit. Constraints that are discovered automatically are often false, and code written by language models can differ from one run to the next [13, 14]. In financial reporting

this matters, because an unjustified rule can produce many alerts on correct data. The research therefore measures what each new rule adds to the controls that are already in use, and at what cost in alerts.

Objective. The objective is to establish when AI helps to maintain and extend expert validation rules, and how a data team can test each proposed rule before relying on it. The experiments use independently published rule changes, planted errors and historical reports. The resulting procedure is implemented in a platform with a review step and a quality gate in a data pipeline: a step that lets a batch of data through, lets it through with a warning, or holds it for review.

2 Pilot work and choice of research direction

The research direction was developed between September and November 2026 through three pilots, a small EBA side study and a test of historical rule execution. The figures below are the author's unpublished results; working reports, scripts and data are in the public pilot repository [15, 17, 11, 18]. Table 4 summarises what each study showed and how it shaped the research questions.

Study	Main finding	Consequence for the thesis
Pilot 1: error detectors on public tables	The best extra detector found 11 to 21 of 40 missed errors, but raised 520 to 900 extra alerts per 800 records.	Always count alerts. Use data that comes with real expert rules.
Pilot 2: language models write financial rules	Model-written versions of DQC rules raised 41% of the official rules' alerts for rules published before the models, and 3–13% for rules published later. The best set of new rules found 6 extra planted errors at 458 alerts per report.	Select rules by what they add to existing rules (RQ2). Control for models having seen the rules before.
SEC corrections and historical test	2 of 20 checked corrections were tagging errors that a rule might catch. 46% of DQC alerts came from rules published after the report was filed.	Build a hand-reviewed set of real errors (dataset D5, Section 5.7). Replay rules as they were at the time (RQ3).
Pilot 3: rules that change with the standard	Simple name replacement solved 10 of 16 simple updates; a small model solved 14. 34 DQC logic changes are not explained by the change files.	Build a benchmark of published expert updates (RQ1), including the harder changes.
EBA side study	A small model predicted which rules would later be deactivated less well overall than simply ranking rules by formula length.	Study changes that have a published answer, not predictions from formula text.

Table 4: Summary of the pilot work (September–November 2026).

2.1 Pilot 1: choosing an error detector

The first question was whether the best error detector depends on how mistakes enter a table. An error detector is a method that flags unusual values without being given explicit rules. The pilot used three public datasets: credit-card customers, bank-marketing records and wine measurements [19, 20, 21]. It changed values on purpose, for example an amount from 250 to 2,500, a removed value or an invalid code. Errors were placed at random, in groups associated with another field, or more often on particular values, following work on realistic error mechanisms [22].

The pilot compared existing rules and statistical checks with seven detectors, including two published error-detection systems, CDC and Raha [23, 24]. The strongest additional detector found 11 to 21 of 40 errors that the other checks missed, but raised 520 to 900 extra alerts in a batch of 800 records. When a reviewer could inspect only 20 alerts per batch, the net benefit, meaning the extra real errors among those 20 alerts, was small or even negative, because the detector's alerts pushed useful alerts of the existing checks out of the review [16]. These datasets also had no independent set of business rules and no verified real errors. The next pilot therefore used reports that come with published expert rules.

2.2 Pilot 2: language models writing financial rules

US listed companies submit financial reports to the Securities and Exchange Commission (SEC). The numbers are labelled with standard accounting names in XBRL. The DQC publishes rules for these reports, together with their code and test reports with the expected results [12, 25]. The pilot asked two small open-weight models, Qwen2.5-Coder-7B and Qwen3-8B [26, 27], to write rules from the descriptions of DQC rules, and to propose new rules for errors that the existing controls missed.

The models often failed to reproduce the official rules. On the 10 DQC rules published before the models were released, the code written by the models raised about 41% of the alerts that the official rules raise on the committee's test reports. On the 6 rules published later, it raised 3% (Qwen2.5-Coder-7B) and 13% (Qwen3-8B) [15]. These groups are small, but the drop is consistent with the models having seen the older rules during training. This is why the thesis controls for prior exposure (Section 6.1).

The pilot also planted errors in 83 altered copies of 12 SEC reports that were set aside for testing. The existing controls (the official DQC rules plus generic and statistical checks) detected 27 of the 83 planted errors. The strongest set of new rules written by a model found six more, but raised on average 458 alerts per report on reports that had never been corrected (measured on 66 reports). The DQC rules raised about 1.8 (measured on 12) [15]. A rule set that finds a few extra errors while flagging hundreds of correct values is difficult to use.

A follow-up looked at individual generated sub-rules. Only sub-rules that caught planted errors missed by the existing rules, at no more than one alert per report, were kept; one sub-rule passed. On reports set aside for evaluation it detected 3 of 195 planted errors that the existing controls missed, at 0.21 alerts per report. A generic filter that ignores the existing rules needed 3.4 alerts per report for the same three detections, and about twice as many alerts as this selection at larger budgets [18]. The gain was small, but it raised a useful question for RQ2: which generated rules add something specific to the checks that are already running?

2.3 SEC corrections and the historical test

SEC data also makes it possible to compare original reports with the amended reports that companies filed later. At first this seemed to provide real errors. A closer review showed why that assumption is unsafe. The author checked 20 correction candidates against the SEC documents, with an AI pre-fill that the author then verified. In nine cases the original number was judged wrong, but seven of these were changes to the company's underlying accounts. Two appeared to be tagging problems: one number carried the wrong accounting name, and one quarterly amount was marked as covering a full year. In three cases the original was not wrong, and eight remained unclear [15, 18]. A rule can detect a wrong name or an inconsistent period. It generally cannot establish that an amount was wrong when the original report accurately reflected the company's accounts at the time. A dataset of real errors therefore needs careful individual review (Section 5.7).

The same data shows how long corrections take to appear. Of the 652 original reports filed in 2023 that were amended by mid-2025, about half were amended within 90 days and 95% within a year [18]. This share is slightly optimistic, because amendments filed after mid-2025 are not in the data. A historical test can therefore treat reports filed more than a year before the corrections are collected as close to complete; more recent reports cannot be treated that way.

A separate test replayed 29 SEC reports filed between February 2023 and March 2025. The official DQC checks ran on 22 of them; five used the 2022 taxonomy, for which the current DQC rule package has no rules, and two downloads failed. Of the 886 DQC alerts, 404 (46%) came from rules that were first published after the report was filed; the test filtered the current rule package by each rule's first release date [17]. Of the 13 hand-checked corrections that could be located in these reports, none was flagged by any rule, expert or AI. Real errors that a rule can detect are rare, and this shapes the expectations for RQ3. A historical evaluation must therefore distinguish what today's rules find from what was possible at the time. This test is the starting point for RQ3.

2.4 Pilot 3: rules that change with the standard

The final pilot (called Pilot B in the working reports) compared the published DQC rule versions for the 2023, 2024, 2025 and 2026 US taxonomies, and EBA formulas across framework releases [11]. Table 5 shows how the DQC sub-rules changed. In 28 of the 62 logic changes, the rule had referred to a name that the new taxonomy deprecated or removed. The change files of the taxonomy explain these cases. The other 34 have other causes, such as corrected calculations or changes in scope.

Taxonomy update	Logic unchanged	Logic changed	of which explained by change files	Added	Removed
2023 → 2024	338	14	1	34	2
2024 → 2025	360	20	7	6	6
2025 → 2026	356	28	20	16	2
Total		62	28	56	10

Table 5: Changes to DQC sub-rules between successive taxonomy years. "Logic unchanged" includes sub-rules in which only internal variable names or the message text changed. "Explained by change files": the old rule used a name that the taxonomy's change files list as deprecated or removed [11].

The pilot first built a simple search method without AI. It searched each rule for names that the new taxonomy lists as deprecated or removed, and flagged the rules that use them. Across the three updates, 23 of the 42 flagged rule files had indeed been changed by the committee, and the search found 23 of the 57 rule files that changed in some way (counted once per annual update). Lists of names that are kept in shared files were not yet traced to the rules that use them, which lowers the second number.

The pilot then tested 16 of the 28 explained changes: those whose old code is at most 3,000 characters, with one model run each. Copying the old rule solved none of them. Replacing old names with their published successors solved 10. Qwen2.5-Coder-7B, given the old rule and a short change notice, solved 14. In four of the extra cases, the old name had been removed without a named successor, and the official rule simply deleted the reference; the model did the same [11]. Section 5.4 shows three of these cases.

These 16 cases are the easiest kind, and the model may have seen newer public rule versions during training; this has not yet been checked. The informative part of the benchmark is the harder

material: the 34 DQC logic changes that the change files do not explain, and EBA formulas that change because a report table changed. In the EBA side study, a small model that read formulas predicted which rules would later be deactivated less accurately overall than simply ranking rules by formula length, a ranking that was only added afterwards; among its top 20 picks it did slightly better [18]. The thesis therefore focuses on rule changes that have a published answer, not on predicting unknown mistakes from formula text.

3 Related research and scientific contribution

This section places the thesis among earlier work. In short: language models have been used to write data-quality rules, and software research has studied how code is updated when its environment changes. No earlier study evaluates rule updates against the updates that experts published, measures what generated rules add beyond existing expert rules, and tests both on historical data in time order.

Error detection and review effort. Research on data cleaning has shown that different detectors find different errors and that extra alerts increase review work. Abedjan et al. compared error detectors on datasets with known errors, and REIN compared cleaning methods more broadly [28, 29]. This supports evaluating the extra value of new checks under a fixed review budget.

Language models that write data-quality rules. Alexander et al. studied the quality and consistency of validation tests generated by language models. Akella et al. used retrieved domain information to generate rules and code, and LeDQeR generated and filtered rules for the Great Expectations library [30, 31, 32]. AuDQ combines language-model agents with rule discovery, documentation and human review [33]. Rule generation is therefore an established topic, and data-quality tools now offer it commercially; at least one such system has been patented [34, 35]. SPADE studies how to select a limited number of checks while controlling false failures, although its checks concern the output of language-model applications [36].

Updating code when its environment changes. CodeUpdateArena and GitChameleon test whether generated code works after a software library has changed, and SWE-bench uses real issues and fixes from software repositories [37, 38, 39]. Continuous data cleaning studies data and constraints that change together [40]. Auto-Validate-by-History learns validation constraints from earlier runs of a data pipeline [41]. These works motivate the use of historical versions, executable tests and later data to evaluate proposed changes. Table 6 compares the most relevant studies.

Contribution. The thesis contributes:

1. a public benchmark of real maintenance tasks for financial validation rules, each consisting of the old rule, the published change to the standard and the experts' updated rule, on which AI and non-AI methods for finding and updating rules are compared;
2. a comparison of rules generated from reporting definitions alone with rules generated after the model has seen errors that the existing rules missed, together with a selection method that keeps only rules that add detections beyond the existing expert rules within a fixed alert budget;
3. a time-ordered evaluation of updated and new rules on real reports, which separates what was possible at the time from what is possible today, and which tests whether a staged approval policy (monitor, warn, block) leads to decisions that still hold on later data;
4. a hand-reviewed set of real errors in SEC reports.

Study	What it does	Compares with existing expert rules	Updates rules when a standard changes	Tests on historical data
Alexander et al. [30]	Generates data tests	No	No	No
Akella et al. [31]	Generates rules and code with retrieved knowledge	No	No	No
LeDQeR [32]	Generates and filters rules	No	No	No
AuDQ [33]	Agents discover rules, with human review	No	No	No
SPADE [36]	Selects checks for language-model output	No	No	No
Auto-Validate-by-History [41]	Learns statistical constraints from past runs	No	No	Yes
CodeUpdateArena [37]	Updates code after library changes	Not applicable	Synthetic changes	No
This thesis	Finds and updates affected rules; generates, selects and replays new rules	Yes	Yes, real published changes	Yes, in filing order

Table 6: Comparison with the most relevant earlier work, as far as reported in the cited sources. The literature review will be updated during the thesis.

4 Research questions

The three research questions follow the workflow in Figure 1: keeping existing rules up to date, adding new rules, and checking both over time.

Main research question. Can open-weight language models help a data team maintain and extend validation rules, and what evidence is needed before a new or changed rule may block data, warn, or only be monitored?

RQ1. When a reporting standard changes, how well can language models find the expert rules that are affected and update them, compared with the updates that the experts published and with simple methods that do not use AI?

RQ2. How many additional errors do AI-generated rules detect beyond the existing expert rules when they are proposed from reporting definitions and example reports alone, and when the model also sees examples of errors that the existing rules missed? How do five methods for selecting among these rules compare at a fixed number of alerts per report?

RQ3. When updated and new rules are run on real reports in the order in which they were filed, and on new reports during the thesis, how many reviewed real errors do they detect, how many alerts do they raise, and do decisions to monitor, warn or block that were taken on earlier reports remain justified on later ones?

	In plain terms	Answer key	Main measures
RQ1	The standard changed. Does AI find the rules that must change, and write the right new version?	The updated rule that the experts published, and the publisher's test reports	Share of flagged rules that really changed; share of changed rules that were flagged; share of updates equal to the expert version or passing its tests
RQ2	Can AI add rules that catch errors the existing rules miss, without too many alerts?	Errors planted at known locations in copies of reports	Extra planted errors caught at 0.5, 1 and 2 alerts per report
RQ3	Do the rules hold up on real reports over time, and are approval decisions still right later?	Real corrections, each reviewed by hand; a hand-checked sample of alerts	Real errors caught; alerts per report; share of approval decisions that still hold

Table 7: The research questions in plain terms. Section 5.8 defines the measures.

5 Datasets and reference material

Each research question needs data together with an answer key that nobody in this project wrote. Five public sources provide this. They are labelled D1 to D5 below.

5.1 How data, rules and AI fit together

Figure 3 shows how the pieces relate. The reporting standard is the shared dictionary. A report uses the names in the dictionary to label its numbers, and a validation rule uses the same names to say which numbers it checks. Software such as Arelle runs the rule on the report and produces alerts. In this study, a language model has one job: it reads definitions, old rules and examples, and writes or edits rule code. That code is then run like any other rule. Only what the executed code reports counts as a result. The model's own explanation of why a rule is correct does not count as evidence.

Table 8 lists the five sources and the research questions they serve.

Source	What is available	Used for
D1, SEC	US company reports, DQC rule versions, FASB taxonomies and official test reports	RQ1, RQ2, RQ3
D2, EBA	European bank report tables and successive versions of validation formulas	RQ1
D3, ESEF	European company reports, IFRS data-quality rules and reference tests	RQ2, RQ3 (preferred extension)
D4, HMDA	US mortgage-application records and published filing checks	RQ2 (optional)
D5, SEC	Original and amended reports, with each change reviewed by hand	RQ3

Table 8: The data sources and the research questions they support. D5 is built during the thesis.

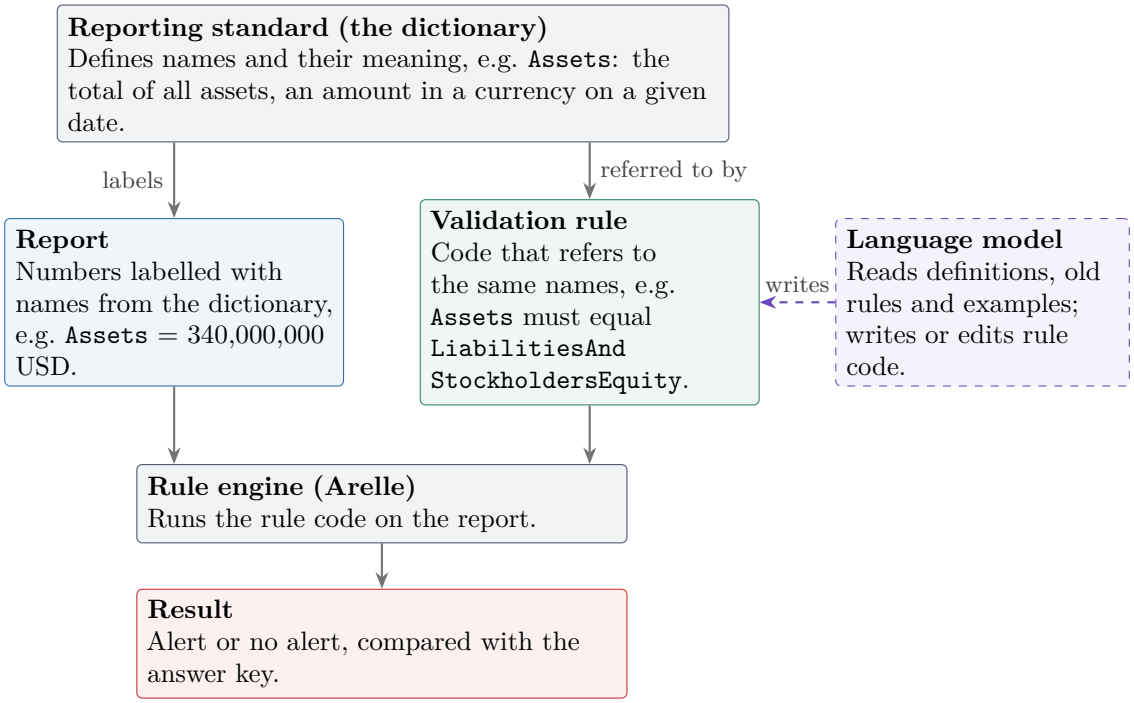


Figure 3: How the reporting standard, reports, rules and the language model relate. The model only proposes rule code; the code is executed and its alerts are evaluated.

5.2 D1: US company reports, DQC rules and FASB taxonomies

The SEC receives financial reports from publicly listed US companies. Each number in these reports is an XBRL fact: a value with its accounting name, its unit and the date or period it refers to [25]. The DQC supplies about 180 expert rules for finding inconsistencies in such reports. The rules are written in XULE, a language for checking XBRL data, and can be run with the open-source processor Arelle [12, 42]. For each rule, the DQC also publishes small test reports together with the messages the rule must produce. These official tests show whether a rule written by someone else, or by a model, behaves like the committee's rule.

D1 is the main source for all three research questions. For RQ1, the input is, for example, a 2025 DQC rule and the changes described in the 2026 taxonomy; the answer is the committee's 2026 version of the rule. For RQ2, a model reads accounting definitions and example reports to suggest rules, and errors are planted in copies of reports to test them. For RQ3, original reports are processed in the order in which they were filed. The SEC Financial Statement Data Sets also show which reports were later followed by an amended version. Between 2023 and mid-2025, the pilot identified 1,294 reports followed by an amended version among 68,875 submissions [15, 25]. For the thesis, these data sets will be extended to the most recent quarter available.

5.3 D2: EBA rules for European bank reporting

European banks report their financial position and risks in standard tables, called templates. The EBA gives each template a code, such as F 01.02, and numbers its rows and columns. A validation rule refers to cells by template, row and column, and relates them in a formula [9, 10]. Figure 4 shows a real rule, `v6053_m`. In words: an amount on the liabilities side of the balance sheet, the provisions for commitments and guarantees that the bank has given (template F 01.02, row 0220, column 0010), must equal the sum of the corresponding amounts in the template for off-balance-sheet exposures (F 09.01.1). That template lists loan commitments, financial guarantees and other commitments given [6].

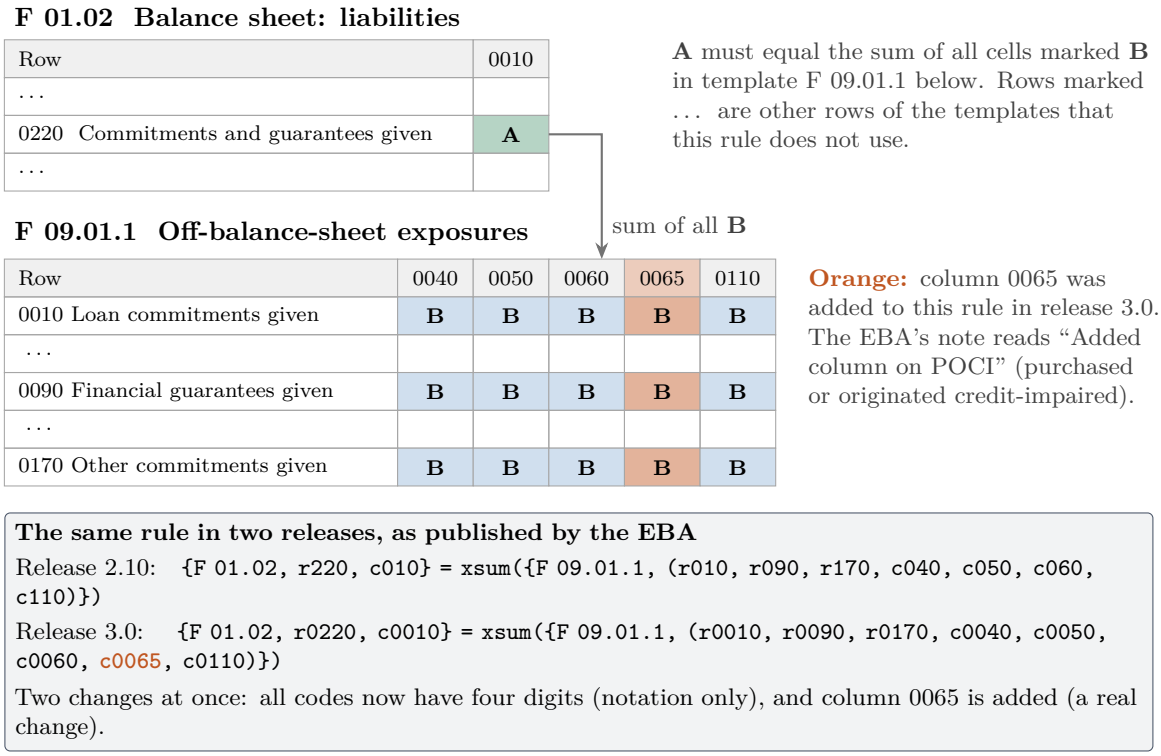


Figure 4: EBA rule `v6053_m`. The grids are simplified excerpts of the two templates; only the rows used by the rule are shown. Formulas and change note from the EBA rule workbooks [9]; row labels from the EBA's Q&A on this rule [6].

When the EBA changes a template, its rows or its definitions, the rules that refer to them may need adjustment. The EBA's workbooks contain the rules of successive releases, with their formulas and a label for the kind of change. In the pilot, 1,406 formula changes were identified across 1,094 rules; 266 of them only changed the notation [11]. For RQ1, a model receives an earlier EBA formula and the relevant changes to the templates or definitions. It proposes the next formula without seeing the version that the EBA later published, which is then used to check the result. The EBA does not publish executable test reports for every rule, so formula comparison and manual checking play a larger role than for the DQC. This dataset is the closest to the supervisory reporting work of a bank.

5.4 The rule-change benchmark built for RQ1

The project combines the DQC and EBA version histories into a set of individual maintenance tasks. Each task records the old rule, the published change to the standard, the updated expert rule and the dates on which each became available. A method sees only the old rule and the change; the expert rule is held back as the answer (Figure 5). The pilot has already assembled the first DQC and EBA comparisons [11].

Table 9 shows three real tasks from the pilot work and how the methods performed on them. They illustrate the range of difficulty. In case A, the change file names the successor, and a simple search-and-replace is enough. In case B, the name was removed without a successor, and the right update is to delete it from a list. In case C, the committee chose a replacement that the change files do not mention, and both methods failed.

The benchmark will also contain changes that cannot be solved by replacing names, such as a calculation corrected by the committee or a rule affected by an EBA template change. These cases show whether a model can use the wider documentation to make the right update. Every

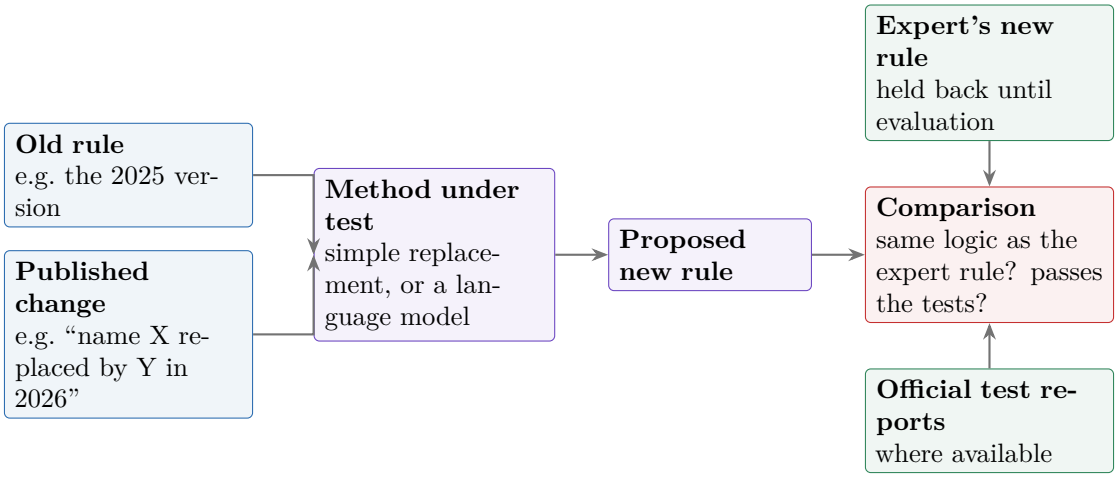


Figure 5: One maintenance task in the benchmark. The method never sees the expert’s new rule.

task is classified as a name change, a logic change or a notation-only change. A sample of the classifications will be checked by a second person before the benchmark is published.

A. A name is replaced by a named successor. DQC_0164, sub-rule 10087, 2025 → 2026	
What the rule checks	Totals such as `Assets` or `Liabilities` must not be split by balance-sheet location; the report should use a more specific item instead.
Published change	`BalanceSheetLocationAxis`: deprecated in 2026; replaced by `StatementOfFinancialPositionLocationBalanceAxis`.
Old code (2025)	`exists({@concept in $HighLevelElements @BalanceSheetLocationAxis = *}#fact2)`
Expert's code (2026)	`exists({@concept in $HighLevelElements @StatementOfFinancialPosition LocationBalanceAxis = *}#fact2)`
Result	Name replacement (no AI): correct. Qwen2.5-Coder-7B: correct.
B. A name is removed without a successor. DQC_0004, sub-rule 9286, 2024 → 2025	
What the rule checks	The total change in cash must equal the sum of the operating, investing and financing cash flows. For the total, the rule takes the first of two names that the report uses.
Published change	`CashAndCashEquivalentsPeriodIncreaseDecrease ExcludingExchangeRateEffect`: removed in 2025; no replacement named.
Old code (2024)	`$base = first-value({nonils @CashCashEquivalentsRestrictedCash...}, {nonils @CashAndCashEquivalentsPeriodIncreaseDecrease ExcludingExchangeRateEffect})`
Expert's code (2025)	`$base = first-value({nonils @CashCashEquivalentsRestrictedCash...})`
Result	Name replacement: not possible, because no successor is named. Qwen2.5-Coder-7B: correct; it deleted the removed name, as the committee did.
C. The experts chose a replacement that the change files do not name. DQC_0046, sub-rule 7480, 2023 → 2024	
What the rule checks	It flags a report whose own calculation places the exchange-rate effect inside net cash from discontinued operations, which the US GAAP taxonomy does not do.
Published change	`EffectOfExchangeRateOnCashAndCashEquivalentsDiscontinuedOperations`: deprecated in 2024; no replacement named.
Old code (2023)	`EffectOfExchangeRateOnCashAndCashEquivalentsDiscontinuedOperations in navigate summation-item descendants from NetCashProvidedByUsedIn DiscontinuedOperations`
Expert's code (2024)	`EffectOfExchangeRateOnCashCashEquivalents RestrictedCashAndRestrictedCashEquivalents DisposalGroupIncludingDiscontinuedOperations in navigate summation-item descendants from ...`
Result	Name replacement: no successor to insert, so the rule stays unchanged (wrong). Qwen2.5-Coder-7B: wrong.

Table 9: Three real maintenance tasks from the pilot work [11]. Only the coloured parts differ between the old and the new code; code shortened with ...; names are the published DQC and US GAAP names. "Correct" means that the logic of the proposal equals the committee's new version after comments and message text are removed.

5.5 D3: European company reports and ESEF rules

Listed companies in the European Union publish their annual financial reports in the European Single Electronic Format (ESEF). Their consolidated financial statements follow International Financial Reporting Standards (IFRS), and the numbers are labelled with Inline XBRL, so that people and software read the same report [43]. XBRL International maintains a public set of additional IFRS data-quality rules with a reference implementation and test suite [44, 45]. These

rules are voluntary and separate from the mandatory ESEF requirements. Table 10 gives an example.

An existing IFRS rule	
Rule	`DQR_IFRS_0080`, "IFRS Non-Negative Items" [46]
What it checks	Values of listed IFRS concepts that are normally positive must not be negative, unless the fact carries one of the listed exceptions, such as a breakdown that allows negative values [47].
Example message	*"Accruals has a value of -400,000 which is less than zero. This element should not have a negative value."* [47]
Moving a rule from US names to IFRS names	
US rule (DQC)	`Assets` must equal `LiabilitiesAndStockholdersEquity`
Same check in IFRS	`Assets` must equal `EquityAndLiabilities`. The meaning is the same, but the names differ, and the rule must still find the right facts.

Table 10: ESEF example. The rule identifier, title and message are published by XBRL International and the DQC; the US-to-IFRS example illustrates the translation task.

D3 provides a second setting for RQ2 and RQ3. The experiment can apply the existing IFRS checks to European reports, let a model propose new rules from the IFRS definitions, plant known errors in copies of reports and measure the extra detections. It also offers a second kind of maintenance task: moving a rule written for US names to the corresponding IFRS names while keeping its intended behaviour. The published IFRS implementation serves as the reference.

5.6 D4: US mortgage applications and HMDA edits (optional)

The Home Mortgage Disclosure Act (HMDA) requires covered US lenders to report information about mortgage applications. Unlike the SEC data, these are rows in a table, one per application, much like an ordinary table in a data platform. Each row contains fields such as the loan amount, the action taken (for example 1 for a loan that was originated) and the location of the property. The US regulators publish filing instructions and checks, which they call *edits* [48]. Table 11 shows three real edits and how a planted error triggers one of them.

Field	Original record	Changed copy	Edit that applies
Loan Amount	250000	250000	V617: "Loan Amount must be a number greater than or equal to 0, and cannot be left blank."
Action Taken	1	1	V618: "Action Taken must equal 1, 2, 3, 4, 5, 6, 7, or 8, and cannot be left blank."
County	19153	19153	
Census Tract	19153001100	19169001100	V627: "The first five digits of the Census Tract must match the reported five-digit County FIPS code." The changed copy fails this edit.

Table 11: HMDA example. Edit numbers and texts are quoted from the 2026 HMDA Filing Instructions Guide [48]; the record values are illustrative. FIPS codes are US federal codes for states and counties; the first five digits of a census tract identify its county.

The public HMDA files have been modified to protect privacy and have already passed the filing

checks, so they contain few visible errors. D4 would therefore use planted errors and the regulator's open-source validator [49]. It is an optional test of RQ2 on data that resembles ordinary tables in a data platform.

5.7 D5: a reviewed dataset of real SEC corrections

D5 will be created during the thesis. When a company files an amended report, the original and the amended version can be compared fact by fact. A changed number is a possible correction, but not necessarily an error that a rule could have found. To decide, a reviewer must read the company's explanation and the context of both facts. Figure 6 shows two real cases from the pilot and how D5 is built.

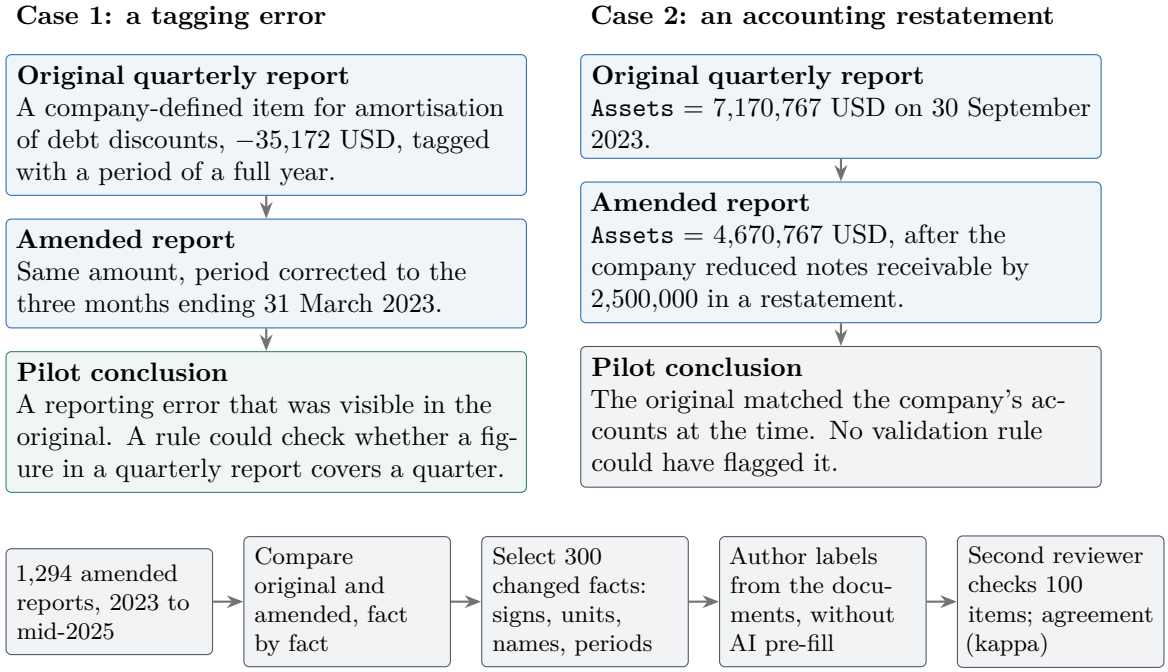


Figure 6: Top: two real correction cases from the pilot review, labelled by the author after an AI pre-fill and checked against the SEC documents [15, 18]; company names omitted. Bottom: how D5 is built.

The target is 300 changed facts, selected across companies and kinds of correction, with emphasis on changes that resemble reporting errors: wrong signs, units, names or periods. For each item the dataset stores the company, the filing dates, the original and amended fact, where the evidence was found, the kind of change, whether the original was wrong, and whether a rule could have detected the problem from the original report alone. The author labels the items from the source documents without an AI pre-fill. A second reviewer independently checks 100 items where possible, and agreement is reported with Cohen's kappa [50]. Unresolved cases remain labelled as uncertain. Table 12 lists the questions a reviewer answers for typical cases.

Expected yield. In the pilot, 2 of 20 checked candidates were reporting errors that a rule might detect, and 8 remained unclear. Even with a selection that favours reporting errors, D5 is therefore likely to contain a few dozen detectable errors rather than hundreds. RQ3 will report its results on D5 with confidence intervals, and every report that contains a D5 item is included in the replay (Section 6.3). D5 is an evaluation dataset; it does not assume that every amended number was an error.

Case	Evidence to inspect	Question for the reviewer
Wrong reporting period	Original and amended XBRL facts, their start and end dates, and any explanatory note	Was a quarterly amount wrongly marked as a full-year amount? Was the error visible in the original report?
Changed balance-sheet amount	Original and amended figures, plus the company's explanation of its accounting correction	Was the original report inconsistent, or did the company later change its underlying accounts?
Same value under a new name	The names in both reports and their definitions	Was the old name wrong, or was the change only a different presentation?

Table 12: How a correction candidate is reviewed. The examples are based on the pilot; final labels require independent checking.

5.8 What counts as a correct result

Each research question has its own reference. In RQ1, the published expert update is the expected output. A model may write different code that behaves the same way, so proposals are also run on the official test reports where they exist, and differences are checked by hand on a sample. A rule that the committee left unchanged may still deserve review; an unchanged rule is therefore a reference decision, not proof that no change was needed. In RQ2, the reference is the exact location of each planted error. Models and selection methods never see the planted errors of the final test; in the documentation-first setting (Section 6.2), the model sees no planted errors at all. Alerts on clean copies are counted as review work, because the original reports are not proven to be error-free. In RQ3, the reference is D5 together with the hand-checked sample of alerts. Table 13 defines the measures.

Question	Measures
RQ1, finding	*Precision*: the share of flagged rules that the experts actually changed. *Recall*: the share of changed rules that were flagged. *Review load*: the number of rule files a person must open.
RQ1, updating	*Exact logic match*: the proposal equals the expert version once comments, message text and internal variable names are removed. *Test pass*: the proposal produces the expected messages on the official test reports (DQC only). *Manual equivalence*: for a sample of non-matching proposals, a person judges whether they behave the same. Reported separately for name changes, logic changes and notation-only changes.
RQ2	*Extra detections*: planted errors caught by the selected rules but not by the existing rules, at budgets of 0.5, 1 and 2 alerts per clean copy. For comparison, the DQC rules raised about 1.8 per report in the pilot. Reported separately for error types seen during development and the error type held back. Also: rules that fail to run, runtime, and variation between repeated model runs.
RQ3	*Real errors caught*: D5 reporting errors flagged, with a confidence interval. *Alert load*: alerts per report. *Alert precision*: the share of real issues in the hand-checked alert sample. *Decisions that hold*: the share of monitor, warn or block decisions that still meet their criteria on later reports (Section 6.3).

Table 13: Measures per research question. All thresholds are registered before the final tests.

6 Experimental design

This section describes the three experiments. In short: RQ1 checks AI-updated rules against the updates experts published; RQ2 checks AI-proposed new rules against errors planted in copies

of reports; RQ3 runs the resulting rules on real reports over time. The terms are explained in Table 1.

6.1 RQ1: finding and updating rules after a change

RQ1 uses two successive versions of the same reporting standard. In the US case, the input consists of the 2025 DQC rules, the 2025 US taxonomy and the published changes for 2026. The task is to find the rules that are probably affected and to write a suitable 2026 version. The actual 2026 DQC rules are kept hidden until evaluation. The same procedure is repeated for the earlier taxonomy years and for pairs of successive EBA releases. Figure 7 shows the two steps.

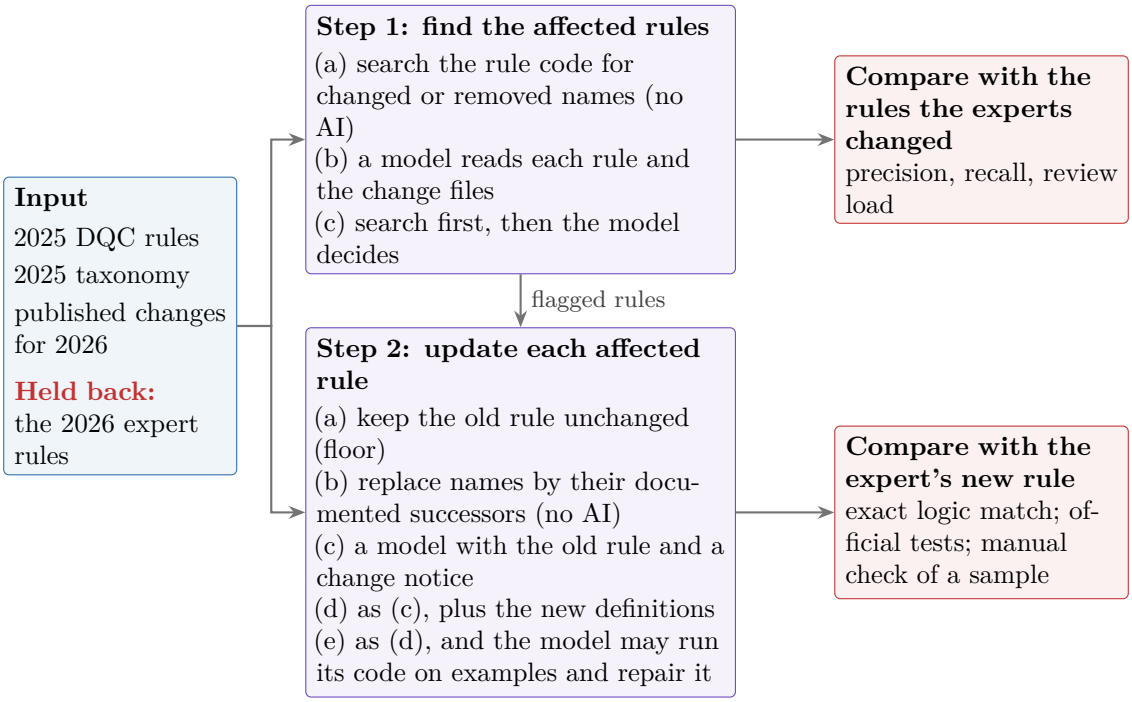


Figure 7: RQ1. Every AI method is compared with a method that uses no AI. Each model setup is repeated five times.

Step 1: finding. A simple program searches the old rule code for names that the new standard changes or removes, including names used through shared lists. A language model receives a rule and the change files and decides whether the rule needs attention. A third method combines the two: the search proposes candidates and the model judges them. For example, an old DQC rule that uses `BalanceSheetLocationAxis` is a candidate, because the 2026 taxonomy replaced that name. We measure how many changed rules each method finds, how many of its flagged rules were actually changed, and how many rule files a person would need to open. An expert may leave a rule unchanged even when it deserves inspection, so this comparison is a reference measure, not a perfect label for every case.

Step 2: updating. Models then write an updated rule in three setups of increasing support, which are compared with keeping the old rule and with replacing each outdated name by its documented successor. In case A of Table 9 replacement is enough; in other cases the right change is to remove a reference or alter a calculation. For the EBA, the output is an updated formula referring to the right cells.

Evaluation. Each proposal is compared with the official update. For the DQC, the official test reports are also run with Arelle to check whether the proposed rule produces the expected messages. For the EBA, formulas are compared after normalising the notation, and differing

proposals are checked by hand. Each model setup is repeated five times to measure consistency. Results are reported separately for name changes, logic changes and notation-only changes.

Prior exposure. Public rule versions may have been part of a model's training data, and the drop from 41% to 3–13% in the second pilot suggests that this matters. The study records when each rule version and each model became available, and compares results on rule versions published before and after the model's release. For OLMo 2, an open model whose training data is public (Section 7), it checks directly whether the rule texts occur in that data [51, 52, 53]. If a new taxonomy year or EBA release appears during the thesis, the methods are also run on it before the experts' updates are published. The remaining risk of prior exposure is reported.

6.2 RQ2: finding gaps and selecting new rules

RQ2 uses D1, with D3 as the preferred second setting. The existing DQC or IFRS rules, together with simple format and statistical checks, form the baseline. The goal is to find out whether AI can suggest reusable rules that detect additional errors. Figure 8 shows the design.

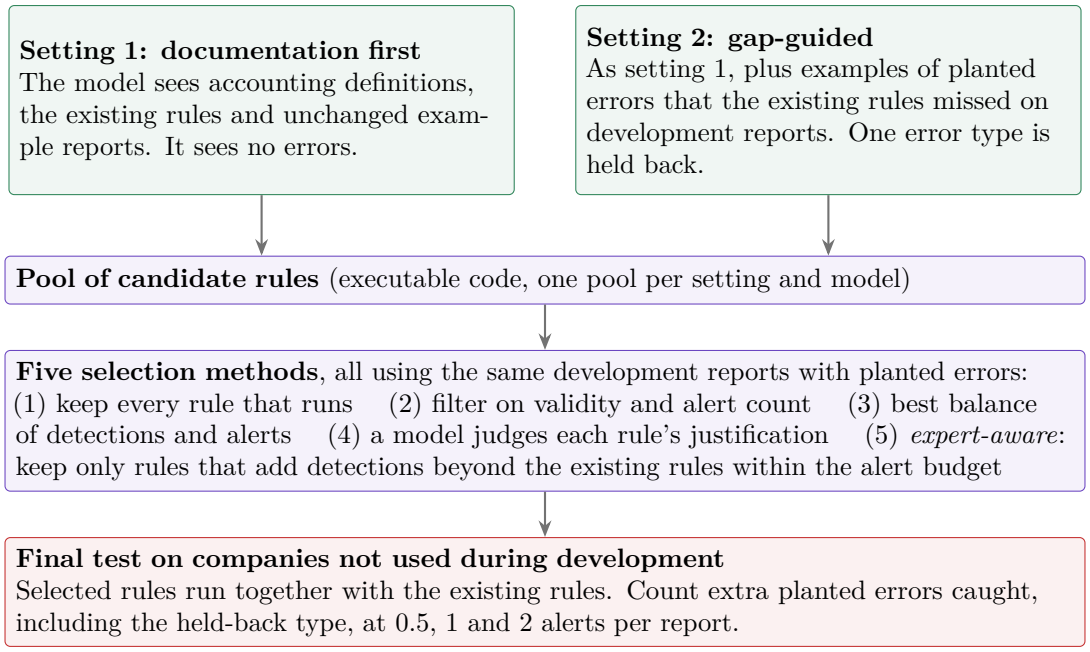


Figure 8: RQ2. The two settings differ only in what the model sees when it writes rules. Selection and testing are the same for both.

Two ways to generate rules. In the *documentation-first* setting, a language model receives accounting definitions, relevant existing rules and example reports. It is asked to propose checks whose meaning can be justified from these materials. It receives no list of missed errors and no examples of planted errors. This represents the realistic situation in which a data team cannot know which errors its current rules miss. A model could, for example, propose a check that compares assets with liabilities and equity. Because `DQC_0004` already does this, such a rule would add nothing. Another candidate might check whether the period attached to a figure is consistent with its definition. Every proposal must be checked against the official documentation.

In the *gap-guided* setting, selected reports are first copied and known facts are deliberately changed (Table 14). The existing rules are run on the changed copies. When a changed fact is not flagged, the development set records a gap. The model receives the relevant definitions and examples of these gaps and proposes rules to close them. In real use, documented past corrections and incidents can provide similar examples, although they cannot reveal every missed error.

Error type	Original fact	Changed copy
Wrong sign	Revenue: \$250,000	Revenue: −\$250,000
Wrong scale	Receivable: \$4,200	Receivable: \$4,200,000
Mistyped digit	Expense: \$31,850	Expense: \$38,150
Wrong period	Q1 expense, January–March	Same value tagged as January–December
Wrong name	An amount reported as an account balance	The same amount under a different accounting name
Missing fact	A required fact is present	The fact is removed from the copy

Table 14: Error types planted in copies of reports, following realistic error models and the pilot [22, 15]. The values are illustrative. The wrong-period type is held back from development and used only in the final test.

Avoiding a circular test. If the model sees examples of exactly the error types that are later used to test it, a good score may only show that it learned the error generator. The design guards against this in three ways. The final test uses companies that were not used during development. One error type, a wrong reporting period, is withheld from development and appears only in the final test; results are reported separately for seen and withheld types, and if time permits each type is withheld in turn. Finally, RQ3 tests the same rules on real corrections, which no generator produced.

Selecting rules. Both settings use the same model families, and each produces a pool of executable candidate rules. Within each pool, five methods select rules using the same development reports: (1) keep every rule that runs; (2) filter on technical validity and alert count, following LeDQeR [32]; (3) choose a small set that balances detections and alerts, inspired by SPADE [36]; (4) ask a model to judge the justification of each rule; and (5) the proposed *expert-aware* method. Method 5 starts from the existing rules. It repeatedly adds the candidate rule that catches the most development errors that neither the existing rules nor the rules already chosen catch, per alert it raises on clean copies, and stops when the alert budget is reached. A rule is only eligible if it meets three conditions. It runs on every development report. It adds detections in at least two of three groups of development companies, so that its value does not depend on a few companies. And it does not simply repeat the alerts of an existing rule. Its thresholds are registered before the final test.

Final test. The final evaluation uses reports from companies that were not used to develop or select the rules. Neither the models nor the selection methods see the planted errors or their locations in this set. Each selected rule runs together with the existing checks. We count the additional planted errors detected at budgets of 0.5, 1 and 2 alerts per report. We also record failed rules, runtime and variation between model runs. Finding ten more planted errors has little practical value if the new rules raise hundreds of extra alerts. A model may also write a rule that catches planted errors for the wrong reason, so the selected rules are checked against the accounting definitions and, where necessary, reviewed by a person.

6.3 RQ3: historical reports, shadow mode and decisions

RQ3 tests the updated rules from RQ1 and the selected new rules from both RQ2 settings on real reports. It replays about 2,000 SEC reports filed from 2023 to the most recent quarter available, in the order in which they were filed. The replay includes every report that contains a D5 item, plus a random sample of other reports. Each alert is stored with the report, the rule, the rule version and three dates.

Three dates. A fair test over time must not use information that was not available at the time. For each alert, three dates matter (Figure 9):

1. *When the rule version was published.* Expert rules are replayed as they stood on each filing date, over the whole period, and compared with replaying today's rules. Older DQC rule packages are available on GitHub; if they cannot be run with the current software, the current rules are filtered by their first release date, as in the pilot. The pilot showed why this matters: 46% of the DQC alert rows came from rules published after the filing date [17].
2. *When the model was released.* A model could not have been used before it existed, and it may have learned from anything published before its release. Approval decisions may use earlier reports as development data, but rules written by a model are only evaluated on reports filed after both the decision date and the model's release date. The release date is the date of the exact model version that is downloaded (its fixed revision on the Hugging Face model hub). With reports up to late 2026, this leaves about two years for a model released in 2024 and a little over a year for a model released in August 2025. Results on earlier reports are reported separately, as a historical stress test.
3. *When a correction became known.* Recent reports have had less time to be amended. Since about 95% of amendments in the pilot data arrived within a year, results based on corrections are reported separately for reports filed in the last year before the corrections are collected.

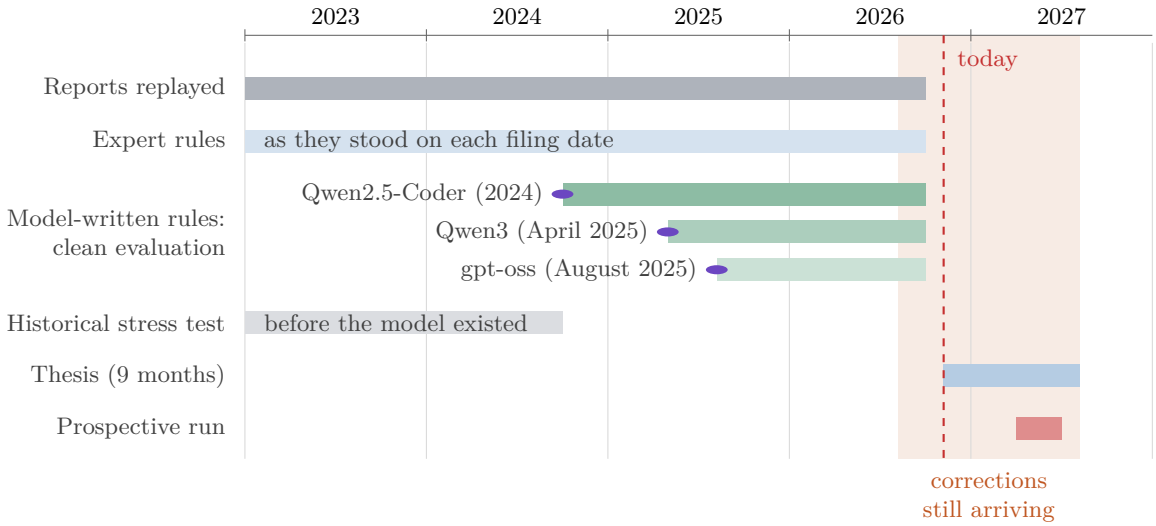


Figure 9: The time design of RQ3 (schematic; thesis months approximate). Expert rules can be replayed for the whole period. Rules written by a model are evaluated only on reports filed after the model's release; earlier results count as a historical stress test. The prospective run uses rules and thresholds that were frozen before any of its reports existed.

Prospective run. Before the final run, the selected rules and the approval thresholds are frozen and stored in the public repository as a dated version that cannot be changed afterwards. They are then run every week on new SEC reports during the thesis, and their alerts are stored before any corrections are known. In this part of the test, nobody can know the outcome in advance, including the author. Only about 2% of reports are later amended (1,294 of 68,875 in the pilot data), about half of the amendments arrive within 90 days, and only a small share of them are reporting errors. The run is therefore expected to find few or no real errors within the thesis. Its main evidence is how frozen rules behave on reports that nobody had seen when the rules were fixed: whether they still run, and how many alerts they raise.

Anticipating expert rules. The DQC added 16 sub-rules in its 2026 version and 6 in its 2025 version. Where a new sub-rule was published after a model's release, according to the repository

history, the study checks whether the rules generated in RQ2 already detected what the new expert rule detects. The number of cases is small, so this is a secondary analysis.

Shadow mode and approval decisions. Every new or updated rule starts in shadow mode: it runs, but its alerts do not stop a report. An approval policy proposes to monitor, warn or block once enough reports have been seen. The proposal depends on whether the rule runs correctly, how often it alerts, the reviewed errors it found, and how stable it is across reporting periods. A person approves or rejects each proposal. The study compares this staged approval with putting rules into their proposed level immediately. A decision counts as *still justified* on later reports when the rule's alert rate stays within the budget for its level and the share of real issues among its reviewed alerts stays above the threshold registered for that level. For example, decisions taken on reports from 2023–2024 are checked on reports from 2025–2026.

Real errors and alert cost. D5 identifies reviewed corrections that were visible in the original report. The replay measures how many of these each rule flags, how often it alerts on reports that were never corrected, and the share of false alarms estimated from the separately reviewed alert sample. An alert on a report that was never corrected is counted as review work; it is not automatically labelled false. A rule that rarely alerts is still unsuitable for blocking data if its business assumption is wrong.

The cost of an outdated rule. Reports for a given taxonomy year are checked with the previous year's rules, with the experts' updated rules and with the model's proposed updates. Differences in the checks that run, the alerts they raise and the errors they find show what an outdated rule misses or adds. This also tests whether the model's update reproduces the behaviour of the expert update. Where an old rule cannot run on a new report at all, that incompatibility is reported directly.

6.4 Professional review and evaluation protocol

The platform produces an evidence card for each proposed rule (Figure 10). The card explains the rule in plain language, shows the old and new code where relevant, names the evidence used, reports the extra detections and alerts, and proposes an action level.

Evidence card (illustrative content; the numbers are not results)

Rule: G-017, "Reported revenue must not be negative unless the report marks it as a correction."
Origin: gap-guided generation, Qwen3-8B, run 3 of 5; chosen by selection method 5.
Overlap with existing rules: no existing DQC rule raises the same alerts on the development reports.

Development reports	**Historical replay**
caught 6 of 9 planted sign errors missed by existing rules	41 alerts on 1,950 reports (0.02 per report)
0.2 alerts per clean copy	3 of 41 alerts match reviewed D5 errors

Proposed action: warn, after three months in shadow mode.
Reviewer: [accept] [revise] [reject] level: [monitor] [warn] [block] time spent: _____ min

Figure 10: Mock-up of an evidence card. Professional reviewers decide on the basis of such cards.

Three to five data-quality or accounting professionals, where available, will each assess about 30 cards. They record whether they would accept, revise or reject a rule, the action level and the time taken. Their decisions are compared with the levels the policy proposed. Ethics approval and the release conditions for annotations will be arranged before the review.

All main evaluation choices, including alert budgets, selection thresholds and approval thresholds, will be documented before the final tests. Development and test companies remain separate, and rule updates are developed on earlier versions and evaluated on later ones. Results will include 95% confidence intervals, with resampling over companies or rules as appropriate, and will report failed runs and negative outcomes. This separates the exploratory pilot results from the final evaluation.

7 Models and baselines

The AI methods use open-weight language models that can run on controlled infrastructure, so no data has to be sent to an external service, the exact model version can be stored, and its release date is known (Table 15). Larger runs use the University of Groningen's Hábrók cluster [54]. Model versions, instructions and generation settings are recorded.

Model	Released	Role
Qwen2.5-Coder 7B and 32B [26]	2024	Specialised in code; the 7B version continues the pilots.
Qwen3 8B and 32B [27]	April 2025	General model; the 8B version continues the pilots.
gpt-oss-20B [55]	August 2025	A second developer, to test whether results depend on one model family.
OLMo 2 7B and 32B [51]	2024–2025	Public training data, which allows a direct check of prior exposure.

Table 15: Planned models. "B" stands for billions of parameters, a rough measure of size. For the time design of RQ3, the release date is the date of the exact model version that is downloaded.

Every AI method has a comparison that does not use AI (Table 16). The models only produce text or code: Arelle executes compatible XBRL rules on the actual reports, and the model's written answer alone never counts as a detection. The five RQ2 selection methods are procedures applied to the same pool of candidate rules. A secondary comparison asks a model to point out suspicious figures in a report directly, without writing a reusable rule. This shows whether writing rules brings a benefit beyond one-off model judgements.

Task	With AI	Without AI
Find affected rules (RQ1)	A model reads each rule and the change files; or search first and the model decides	Search the rule code for changed or removed names
Update a rule (RQ1)	A model with a change notice, with new definitions, or with the option to run and repair its code	Keep the old rule; replace names by their documented successors
Add rules (RQ2)	Documentation-first or gap-guided generation, followed by selection	The existing expert rules plus inexpensive statistical checks
Flag errors directly (RQ2, secondary)	A model lists suspicious facts in a report	—

Table 16: Each AI method and its comparison without AI.

8 Engineering artefact and intended use

The engineering deliverable is a platform for preparing, testing and approving data-quality rules, whether they were written by people or by a model. Figure 11 shows how it is used. A *rule register* stores each rule's code, origin, standard version and approval status. A *change radar* identifies the rules affected by new definitions. An *update assistant* proposes modified code and shows the difference with the old rule. A *stress test* plants known errors in copies of data to show which errors the current rules miss, and a *rule generator* proposes rules for those gaps. A *selection* step keeps candidates that add useful detections at an acceptable alert level. A *backtest* runs the selected rules on historical reports. The reviewer sees the evidence card and records a decision.

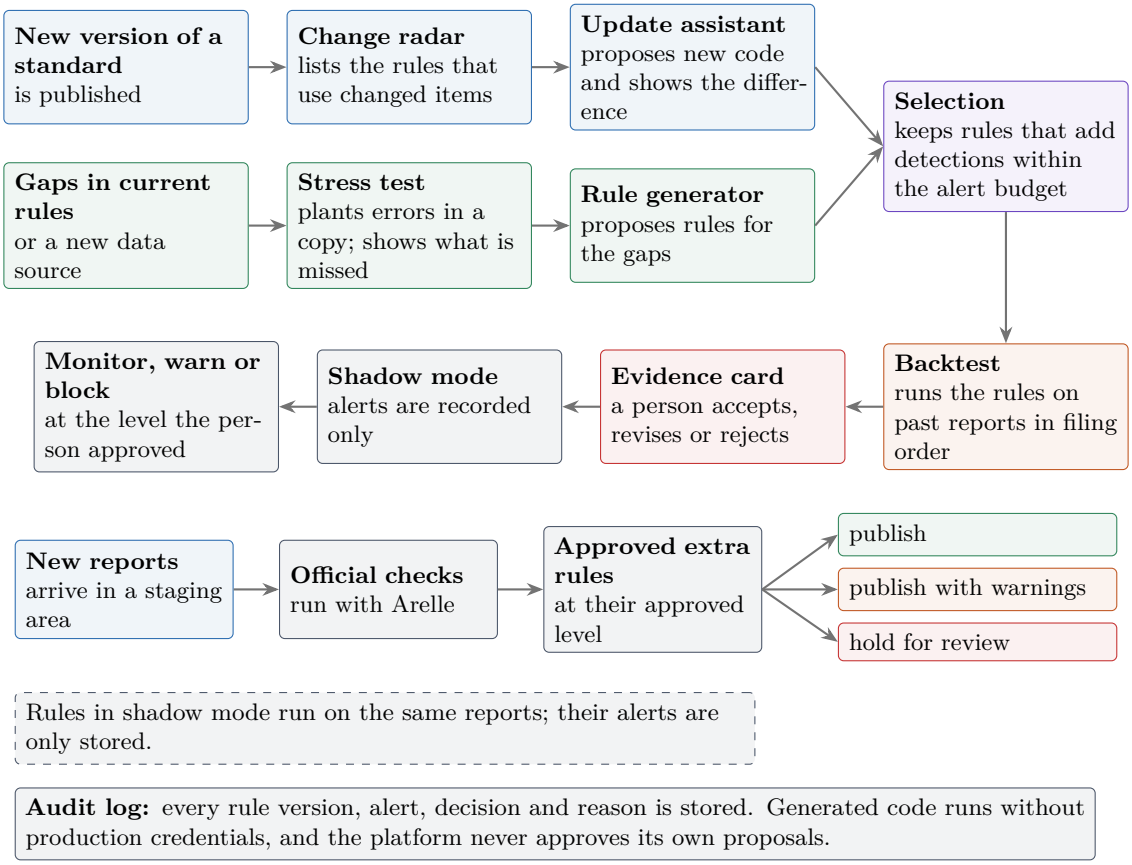


Figure 11: Top: how the platform is used to update and add rules. Bottom: the quality gate in the demonstration data pipeline.

The demonstration pipeline uses Apache Spark and Delta Lake, open-source software for processing and storing large tables [56, 57]. New reports enter a staging area. Arelle runs the official XBRL checks, the approved additional rules run next, and the batch is published, published with warnings or held, according to the levels of the rules that fired. Rules in shadow mode see the same data without affecting the decision. Generated code runs without production credentials, and the platform does not approve its own proposals. The reviewer and an audit log remain part of the process.

The public DQC and EBA rule sets provide the test cases for the research. In a bank, the same procedure could be applied to locally written rules that depend on changing reporting definitions or source data. This is the intended organisational use; the experiments do not claim to measure Rabobank's current maintenance costs or production error rates. The platform would still need internal security approval and integration work before production use. To keep the work manageable, the review screen will be simple and the pipeline will be demonstrated on SEC data only.

9 Plan, scope and expected outputs

The project runs for nine months (Table 17). Progress is reviewed at the end of month 3.

Months	Work
1	Evaluation protocol and registration of thresholds; literature update; ethics request; cluster access; extend the SEC data to the most recent quarter; DQC and EBA rule-change benchmark; rule register
2–3	RQ1 on DQC and EBA (finding and updating); dating of rule versions and models for the exposure check; start of the D5 reviews; progress review at the end of month 3
4–5	RQ2 on SEC data, both settings and the five selection methods; ESEF replication; data pipeline with quality gate and shadow mode; at the end of month 5, freeze and tag the selected rules and thresholds
6–7	RQ3: historical replay, clean evaluation windows and the cost of outdated rules; start of the prospective run at the beginning of month 6; completion of D5 and the alert sample; evidence cards and professional review
8	Prospective run continues; analysis across datasets; robustness checks; publication of the benchmark; demonstration to Avanade and Rabobank
9	Final collection of corrections for the prospective run; writing, documentation and contingency

Table 17: Nine-month plan.

Minimum scope. The minimum scientific scope is RQ1 on the DQC and EBA rules; RQ2 on SEC data with both generation settings and the five selection methods; RQ3 on SEC data, including the clean evaluation windows and a prospective run of at least two months; at least two model families; and the double-reviewed part of D5. ESEF (D3) is the preferred extension and HMDA (D4) is optional. If the programme becomes too large, the work is reduced in this order: HMDA, then the depth of the ESEF analysis, then the number of models, then the length of the prospective run. The held-out evaluation is not reduced.

Expected outputs. The expected outputs are a reproducible comparison of AI and non-AI rule maintenance; a public benchmark of expert rule changes; an evaluation of two ways to generate new rules and five selection methods at fixed alert budgets; a time-ordered and prospective evaluation of the resulting rules; a reviewed set of real SEC corrections, subject to permissions; and a platform that demonstrates testing, approval and historical replay. The conclusions may show that simple methods are sufficient for many updates, or that generated rules add little beyond the expert controls. These are valid research outcomes when they are supported by the final experiments. The thesis will distinguish pilot evidence, official reference results and newly verified findings.

Risk	Response
Simple methods already solve most rule updates	Report where the boundary lies; AI is evaluated on the remaining cases, which shows a team where AI is worth using.
The experts' update is not the only correct one	Run proposals on test reports where possible; judge a sample of differing proposals by hand.
Models saw the expert updates during training	Compare rule versions published before and after each model's release; check OLMo's public training data; test on a new release if one appears.
Gap-guided rules only learn the error generator	Test on companies not used in development; hold back one error type; check the same rules on real corrections in RQ3.
Few reviewed real errors	Favour reporting-error types when selecting D5; include every D5 report in the replay; report confidence intervals.
Reports that were never corrected are treated as correct	Count their alerts as review work; estimate false alarms from the reviewed alert sample.
The historical test uses information from the future	Store three dates per alert; evaluate model-written rules only after the model's release; add a prospective run with frozen rules.
Recent reports have not yet been corrected	Report correction-based results separately for reports filed in the last year before the corrections are collected; treat the prospective run mainly as evidence on alert load.
Older DQC rule packages do not run with current software	Filter the current rules by their first release date, as in the pilot, and report this limitation.
Building the pipeline takes time from the research	Keep the pipeline small and limited to SEC data; the scientific results come from RQ1 to RQ3.
Professional reviewers are not available in time	Request ethics approval in month 1; reduce the number of cards; report the limitation if no independent review is possible.

Table 18: Main risks and responses.

References

[1] Basel Committee on Banking Supervision. Principles for effective risk data aggregation and risk reporting (BCBS 239). January 2013. https://www.bis.org/publ/bcbs239.htm.

[2] European Central Bank. Guide on effective risk data aggregation and risk reporting. May 2024. https://doi.org/10.2866/081671.

[3] European Central Bank. ECB Annual Report on supervisory activities 2024, Section 1.2.3.2: Risk data aggregation and risk reporting. 2025. https://www.bankingsupervision.europa.eu/press/other-publications/annual-report/html/ssm.ar2024~700cba1314.en.html.

[4] L. Tran. Correspondence with the author, 4 September 2026. Unpublished; observations specific to his working context.

[5] XBRL US Data Quality Committee. Rule DQC_0004: Element Values Are Equal, including the official message example. https://xbrl.us/data-rule/dqc_0004/.

[6] European Banking Authority. Single Rulebook Q&A 2022_6353, validation rule v6053_m, final answer 31 March 2023. https://eba.europa.eu/single-rule-book-qa/qna/view/publicId/2022_6353.

[7] Financial Accounting Standards Board. US GAAP Financial Reporting Taxonomy, releases 2023 to 2026, including the deprecated-concepts files. https://xbrl.fasb.org/us-gaap/.

[8] Financial Accounting Standards Board. 2026 GAAP Financial Reporting Taxonomy and Data Quality Committee Rules Taxonomy Technical Guide. 2026. https://xbrl.fasb.org/resources/annualrelease/2026/GAAP_Financial_Reporting_Taxonomy_and_Data_Quality_Committee_

Rules_Taxonomy_Technical_Guide.pdf.

[9] European Banking Authority. Validation rules, rule workbooks for the reporting framework releases up to 3.5, including change labels and deactivation history. https://www.eba.europa.eu/risk-and-data-analysis/reporting/reporting-frameworks/validation-rules-packages.

[10] European Banking Authority. DPM data dictionary. https://www.eba.europa.eu/risk-and-data-analysis/reporting/dpm-data-dictionary.

[11] T. Oosterbaan. Pilot B: rule maintenance when a data standard changes. Project working report, masterthesis_direction3_pilot, docs/pilot_b_rule_maintenance.md, November 2026. Unpublished.

[12] XBRL US Data Quality Committee. DQC US rules, documentation and reference tests, version 31. Official repository, accessed November 2026. https://github.com/DataQualityCommittee/dqc_us_rules.

[13] A. Martin, E. C. de Almeida, O. Romero, and A. Queralt. How and why false denial constraints are discovered. Proceedings of the VLDB Endowment, 18(10):3477–3489, 2025. https://doi.org/10.14778/3748191.3748209.

[14] S. Ouyang, J. M. Zhang, M. Harman, and M. Wang. An empirical study of the non-determinism of ChatGPT in code generation. ACM Transactions on Software Engineering and Methodology, 34(2), 2025. https://doi.org/10.1145/3697010.

[15] T. Oosterbaan. Pilot report: AI-written data-quality rules versus expert controls on SEC XBRL filings. Project working report, masterthesis_direction3_pilot, docs/pilot_report.md, October 2026. Unpublished; numerical results reported by the author.

[16] T. Oosterbaan. First pilot: code, configurations and results. Repository masterthesis_pilot_final, 2026. Private.

[17] T. Oosterbaan. Backtest feasibility check. Project working report, masterthesis_direction3_pilot, docs/backtest_feasibility.md, November 2026. Unpublished.

[18] T. Oosterbaan. SEC rule-generation pilot, follow-up selection test, EBA side study, rule-maintenance pilot and amendment timing: code, results and reports. Public repository https://github.com/oosterbaanvu/masterthesis_direction3_pilot, accessed 9 November 2026.

[19] I.-C. Yeh. Default of credit card clients. UCI Machine Learning Repository, 2009. https://doi.org/10.24432/C55S3H.

[20] S. Moro, P. Rita, and P. Cortez. Bank marketing. UCI Machine Learning Repository, 2014. https://doi.org/10.24432/C5K306.

[21] P. Cortez, A. Cerdeira, F. Almeida, T. Matos, and J. Reis. Wine quality. UCI Machine Learning Repository, 2009. https://doi.org/10.24432/C56S3T.

[22] P. Jung, S. Jäger, N. Chandler, and F. Biessmann. Towards realistic error models for tabular data. ACM Journal of Data and Information Quality, 17(4), 2025. https://doi.org/10.1145/3774914.

[23] S. Jäger and F. Biessmann. From data imputation to data cleaning: automated cleaning of tabular data improves downstream predictive performance. AISTATS, PMLR 238:3394–3402, 2024.

[24] M. Mahdavi et al. Raha: a configuration-free error detection system. SIGMOD, 865–882, 2019. https://doi.org/10.1145/3299869.3324956.

[25] US Securities and Exchange Commission. Financial Statement Data Sets, 2023q1 to 2025q2. https://www.sec.gov/data-research/sec-markets-data/financial-statement-data-sets.

[26] B. Hui et al. Qwen2.5-Coder technical report. arXiv:2409.12186, 2024.

[27] A. Yang et al. Qwen3 technical report. arXiv:2505.09388, 2025.

[28] Z. Abedjan et al. Detecting data errors: where are we and what needs to be done? Proceedings of the VLDB Endowment, 9(12):993–1004, 2016. https://doi.org/10.14778/2994509.2994518.

[29] M. Abdelaal, C. Hammacher, and H. Schöning. REIN: a comprehensive benchmark framework for data cleaning methods in ML pipelines. EDBT, 2023. https://doi.org/10.48786/edbt.2023.43.

[30] R. Alexander et al. Evaluating the decency and consistency of data validation tests generated by LLMs. arXiv:2310.01402, 2023.

[31] A. Akella, A. Kaul, K. Narayanam, and S. Mehta. Quality assessment of tabular data using large language models and code generation. EMNLP Industry Track, 2025. https://doi.org/10.18653/v1/2025.emnlp-industry.183.

[32] A.-C. Glock et al. Data quality rule generation with LLMs. arXiv:2609.06053, September 2026.

[33] J. Biggiogera et al. AuDQ: auditable data quality through agentic rule discovery. EMNLP Industry Track, 2026. https://aclanthology.org/2026.emnlp-industry.113/.

[34] T. Rehberger, T. Hütter, L. Ehrlinger, and W. Wöß. Evaluating data quality tools: measurement

capabilities and LLM integration. arXiv:2604.09163, 2026.
[35] Systems and methods for automatically ensuring data quality using LLMs. US Patent 12,632,447.
[36] S. Shankar et al. SPADE: synthesizing data quality assertions for large language model pipelines. Proceedings of the VLDB Endowment, 17(12), 2024. https://doi.org/10.14778/3685800.3685835.
[37] Z. L. Liu, S. Pandit, X. Ye, E. Choi, and G. Durrett. CodeUpdateArena: benchmarking knowledge editing on API updates. arXiv:2407.06249, 2024.
[38] D. Misra, N. Islah, V. May, B. Rauby, Z. Wang, J. Gehring, A. Orvieto, M. Chaudhary, E. B. Muller, I. Rish, S. E. Kahou, and M. Caccia. GitChameleon 2.0: evaluating AI code generation against Python library version incompatibilities. arXiv:2507.12367, 2025.
[39] C. E. Jimenez, J. Yang, A. Wettig, S. Yao, K. Pei, O. Press, and K. Narasimhan. SWE-bench: can language models resolve real-world GitHub issues? ICLR, 2024. arXiv:2310.06770.
[40] M. Volkovs, F. Chiang, J. Szlichta, and R. J. Miller. Continuous data cleaning. ICDE, 244–255, 2014. https://doi.org/10.1109/ICDE.2014.6816655.
[41] D. Tu et al. Auto-Validate by-History: auto-program data quality constraints to validate recurring data pipelines. KDD, 2023. https://doi.org/10.1145/3580305.3599776.
[42] Arelle. Open-source XBRL processor. https://arelle.org.
[43] European Securities and Markets Authority. Enforcement of corporate reporting: ESEF reporting. https://www.esma.europa.eu/issuer-disclosure/enforcement-corporate-reporting.
[44] XBRL International. ESEF Data Quality Rules. https://esef-dqr.xbrl.org.
[45] XBRL International. ESEF DQR conformance suite. https://gitlab.xbrl.org/xii/dqr-conformance.
[46] XBRL International. ESEF Data Quality Rules and Guidance, including DQR_IFRS_0080 and DQR_IFRS_0102. https://esef-dqr.xbrl.org/introduction-to-dqr/rules-guidance/.
[47] XBRL US Data Quality Committee. DQC_IFRS_0080: IFRS non-negative items, rule documentation. https://github.com/DataQualityCommittee/dqc_us_rules/blob/master/docs/ESEF/DQC_IFRS_0080/DQC_0080.md.
[48] FFIEC and CFPB. 2026 HMDA Filing Instructions Guide. https://ffiec.cfpb.gov/documentation/fig/2026/overview.
[49] Consumer Financial Protection Bureau. HMDA Platform, open-source filing and validation software. https://github.com/cfpb/hmda-platform.
[50] J. Cohen. A coefficient of agreement for nominal scales. Educational and Psychological Measurement, 20(1):37–46, 1960.
[51] Team OLMo. 2 OLMo 2 Furious. arXiv:2501.00656, 2024.
[52] J. Liu et al. OLMoTrace: tracing language model outputs back to trillions of training tokens. ACL System Demonstrations, 2025. arXiv:2504.07096.
[53] N. Jain et al. LiveCodeBench: holistic and contamination free evaluation of large language models for code. ICLR, 2025. arXiv:2403.07974.
[54] University of Groningen. Hábrók cluster documentation: Apptainer containers. https://wiki.hpc.rug.nl/habrok/examples/apptainer.
[55] OpenAI. gpt-oss-120b and gpt-oss-20b model card. arXiv:2508.10925, 2025.
[56] M. Zaharia et al. Apache Spark: a unified engine for big data processing. Communications of the ACM, 59(11):56–65, 2016. https://doi.org/10.1145/2934664.
[57] M. Armbrust et al. Delta Lake: high-performance ACID table storage over cloud object stores. Proceedings of the VLDB Endowment, 13(12):3411–3424, 2020. https://doi.org/10.14778/3415478.3415560.